B.E. / B.TECH. – FOURTH SEMESTER

CS3591 – COMPUTER NETWORKS

PRACTICAL RECORD – 20 EXPERIMENTS

Anna University – Regulations 2021

Name of the Student	.....
Register Number	.....
Department / College	.....
Academic Year	.....

Based on the uploaded CS3591 Computer Networks practical question paper (20 questions).

Exam pattern shown in the question paper

Aim / Principle / Apparatus / Procedure	Program / Drawing / Tabulation	Calculation & Results	Viva	Record	Total
20	30	30	10	10	100

How to Use This Record

This record follows the 20 questions in the uploaded CS3591 question paper. Each experiment contains Aim, Principle, Apparatus Required, Procedure, Program or simulator workflow, Sample Output, Calculation, Result, and Viva-Voce questions.

Important notes

1. Replace sample outputs and calculated values with the results from your own lab execution where required.
2. Questions that explicitly require NS or a network simulator use the simulator's native script format (for example, Tcl in NS-2), not Java. The Java programs are provided for socket and algorithm experiments where Java is suitable.
3. ARP/RARP/DNS examples are educational simulations with local tables; they are not complete implementations of the real network protocols.
4. Use Wireshark only on your own traffic or an instructor-authorized lab network.

Compilation and Execution – Java

For a single Java file: save it using the public class name, compile with javac FileName.java, then run with java FileName. For client-server experiments, open two terminals, start the server first, and then start the client. Use the same port number in both programs.

Contents

Exp.	Experiment
1	TCP Client–Server File Transfer and HTTP Webpage Download
2	TCP Client–Server Chat Application Using Sockets
3	Hello Server Using TCP
4	Simulation of DNS Using UDP Socket
5	TCP File Transfer – User-Requested File Download and FTP Credential Capture
6	Reverse Address Resolution Protocol (RARP) Using UDP
7	Distance Vector Routing – Least-Cost Route
8	Congestion Control Mechanism Simulation Using Network Simulator
9	Sliding Window Throughput with Packet Size, Error Rate and RTT
10	Comparison of Distance Vector and Link State Routing
11	Error Detection and Correction Techniques
12	CRC Error Detection Between Source and Destination
13	Simulation of ARP Protocol for Server and Client
14	Datagram Socket Client/Server Messaging Using UDP
15	Client Program for Simple Mail Transfer Protocol (SMTP)
16	RARP Protocol Simulation for Server and Client
17	HTTP Web Client Using TCP Sockets and Network-Layer Packet Investigation
18	TCP/UDP Performance Analysis Using Simulation and FTP Capture
19	HDLC Frame – Bit Stuffing and Character Stuffing
20	TCP Server Reads a String, Displays Day/Time and Replies to Client; TCP Performance

EXPERIMENT 01: TCP Client–Server File Transfer and HTTP Webpage Download

AIM

To develop a TCP client-server application to transfer a file. Also, to download a webpage through a TCP socket and save the received content as an HTML file.

PRINCIPLE

TCP is connection-oriented and provides reliable, ordered byte-stream delivery. A server listens on a port; a client connects, and both exchange data through input/output streams. HTTP is an application-layer protocol that commonly runs over TCP port 80.

APPARATUS REQUIRED

Computer with JDK 8 or later, terminal/IDE, two terminal windows, and internet/LAN connection for the HTTP part.

PROCEDURE

1. Create a server socket and wait for a client connection.

2. The client connects to the server using the server IP address and port number.

3. The client sends the requested file name.

4. The server checks the file and sends its bytes through the TCP output stream.

5. The client writes received bytes into a new file.

6. For the HTTP part, connect to a web server on port 80, send an HTTP GET request, read the response, and save the HTML body.

PROGRAM / SIMULATOR WORKFLOW

```
// TCP file transfer: save as FileServer.java
import java.io.*;
import java.net.*;
public class FileServer {
  public static void main(String[] args) throws Exception {
    try (ServerSocket ss = new ServerSocket(5000)) {
      System.out.println("Server waiting...");
      try (Socket s = ss.accept();
           DataInputStream in = new DataInputStream(s.getInputStream());
           DataOutputStream out = new DataOutputStream(s.getOutputStream())) {
        String name = in.readUTF();
        File f = new File(name);
        if (!f.exists()) { out.writeLong(-1); return; }
        out.writeLong(f.length());
        try (FileInputStream fin = new FileInputStream(f)) {
          byte[] b = new byte[4096]; int k;
          while ((k = fin.read(b)) != -1) out.write(b, 0, k);
        }
        System.out.println("Sent: " + name);
      }
    }
  }
}
// Client: save as FileClient.java
import java.io.*;
import java.net.*;
public class FileClient {
  public static void main(String[] args) throws Exception {
    try (Socket s = new Socket("localhost", 5000);
         DataOutputStream out = new DataOutputStream(s.getOutputStream());
         DataInputStream in = new DataInputStream(s.getInputStream())) {
      out.writeUTF(args.length > 0 ? args[0] : "sample.txt");
      long size = in.readLong();
      if (size < 0) { System.out.println("File not found"); return; }
      try (FileOutputStream f = new FileOutputStream("received_file")) {
        byte[] b = new byte[4096]; long left = size;
        while (left > 0) {
          int k = in.read(b, 0, (int)Math.min(b.length, left));
          if (k < 0) break; f.write(b, 0, k); left -= k;
        }
      }
      System.out.println("File received successfully");
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Server: Server waiting... then Sent: sample.txt. Client: File received successfully. The received_file contains the transferred bytes.

TABULATION / CALCULATION

Transferred bytes = original file size in bytes. Transfer time = end time – start time. Throughput = (file size in bits) / transfer time (seconds).

RESULT

Thus, a file was transferred between a TCP client and server. The same TCP connection method can be used to request an HTTP page and store the response as an HTML file.

VIVA-VOCE

Question	Answer
What is TCP?	A connection-oriented, reliable transport-layer protocol.
What is a socket?	An endpoint used by programs to communicate over a network.
What is a port number?	A number used to identify an application/service on a host.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: HTTP download note: connect to host port 80, send `GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n`, read until the server closes the connection, and save the response. Separate HTTP headers from the body before saving only the HTML.

EXPERIMENT 02: TCP Client–Server Chat Application Using Sockets

AIM

To implement a two-way chat application between a TCP client and server using Java sockets.

PRINCIPLE

TCP establishes a reliable connection. The server accepts a client, and both sides exchange text messages using character streams. A line of text is sent at a time.

APPARATUS REQUIRED

Two computers or two terminal windows, JDK 8+, and LAN/localhost.

PROCEDURE

1. Run the server first; it waits on port 5001.

2. Run the client and connect to the server IP and port.

3. Type a message at either side and press Enter.

4. The other side receives and displays the message.

5. Type `bye` to end the chat.

PROGRAM / SIMULATOR WORKFLOW

```
// ChatServer.java
import java.io.*;
import java.net.*;
public class ChatServer {
  public static void main(String[] a) throws Exception {
    try (ServerSocket ss = new ServerSocket(5001);
         Socket s = ss.accept();
         BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
         PrintWriter out = new PrintWriter(s.getOutputStream(), true);
         BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
      String msg;
      while ((msg = in.readLine()) != null) {
        System.out.println("Client: " + msg);
        if (msg.equalsIgnoreCase("bye")) break;
        System.out.print("Server: ");
        msg = kb.readLine(); out.println(msg);
        if (msg.equalsIgnoreCase("bye")) break;
      }
    }
  }
}
// ChatClient.java
import java.io.*;
import java.net.*;
public class ChatClient {
  public static void main(String[] a) throws Exception {
    try (Socket s = new Socket("localhost", 5001);
         BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
         PrintWriter out = new PrintWriter(s.getOutputStream(), true);
         BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
      while (true) {
        System.out.print("Client: ");
        String msg = kb.readLine(); out.println(msg);
        if (msg.equalsIgnoreCase("bye")) break;
        String reply = in.readLine();
        if (reply == null) break;
        System.out.println("Server: " + reply);
        if (reply.equalsIgnoreCase("bye")) break;
      }
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Client and server exchange messages alternately. Typing bye terminates the chat session.

TABULATION / CALCULATION

Number of messages exchanged = count of sent lines. Average message delay = total measured round-trip time / number of test messages.

RESULT

Thus, a TCP-based client-server chat application was implemented successfully.

VIVA-VOCE

Question	Answer
Why is TCP suitable for chat?	It provides reliable, ordered delivery.
What does accept() do?	It waits for and accepts an incoming client connection.
What is localhost?	The loopback address referring to the same computer.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Run: `javac ChatServer.java ChatClient.java`; terminal 1: `java ChatServer`; terminal 2: `java ChatClient`.

EXPERIMENT 03: Hello Server Using TCP

AIM

To create a TCP server that receives the string “Hello, world!” from a client and displays it.

PRINCIPLE

A TCP client connects to a listening server. The client writes a message to the socket output stream; the server reads it from the input stream.

APPARATUS REQUIRED

Computer, JDK 8+, and terminal/IDE.

PROCEDURE

1. Compile the server and client programs.

2. Start the server on port 5002.

3. Start the client; it connects to localhost:5002.

4. The client sends Hello, world! and closes the connection.

5. The server displays the received message.

PROGRAM / SIMULATOR WORKFLOW

```
// HelloServer.java
import java.io.*;
import java.net.*;
public class HelloServer {
  public static void main(String[] args) throws Exception {
    try (ServerSocket ss = new ServerSocket(5002)) {
      System.out.println("Waiting for client...");
      try (Socket s = ss.accept();
           BufferedReader in = new BufferedReader(
             new InputStreamReader(s.getInputStream()))) {
        System.out.println("Received: " + in.readLine());
      }
    }
  }
}
// HelloClient.java
import java.io.*;
import java.net.*;
public class HelloClient {
  public static void main(String[] args) throws Exception {
    try (Socket s = new Socket("localhost", 5002);
         PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
      out.println("Hello, world!");
      System.out.println("Message sent");
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Server: Waiting for client... Received: Hello, world! Client: Message sent.

TABULATION / CALCULATION

Message length = number of characters in “Hello, world!” = 13 characters (including the comma and space).

RESULT

Thus, the TCP client connected to the server, sent the required message, and closed the connection.

VIVA-VOCE

Question	Answer
Which port is used in this program?	TCP port 5002, chosen by the programmer.
What is ServerSocket?	A Java class that listens for incoming TCP connections.
What happens when the client closes?	The socket connection is closed and the server can finish reading.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Compile and run the server before running the client. If the port is busy, choose another unused port in both files.

EXPERIMENT 04: Simulation of DNS Using UDP Socket

AIM

To simulate a simple DNS request-response system using UDP datagram sockets and a user-specified frame/message size.

PRINCIPLE

DNS translates a domain name into an IP address. UDP is connectionless and sends independent datagrams. This lab program uses a small local mapping to demonstrate the request and reply; it is not a full DNS implementation.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, and two terminal windows.

PROCEDURE

1. Start the UDP DNS server on port 5003.

2. The client asks the user for a domain name and frame size.

3. The client sends the domain name as a UDP datagram.

4. The server looks up the name in its local table and sends an IP address or not-found reply.

5. The client displays the reply.

PROGRAM / SIMULATOR WORKFLOW

```
// DNSServer.java
import java.net.*;
public class DNSServer {
  public static void main(String[] a) throws Exception {
    try (DatagramSocket ds = new DatagramSocket(5003)) {
      byte[] b = new byte[1024];
      DatagramPacket p = new DatagramPacket(b, b.length);
      ds.receive(p);
      String domain = new String(p.getData(), 0, p.getLength()).trim();
      String ip = domain.equals("example.com") ? "93.184.216.34" : "NOT FOUND";
      byte[] r = ip.getBytes();
      ds.send(new DatagramPacket(r, r.length, p.getAddress(), p.getPort()));
    }
  }
}
// DNSClient.java
import java.net.*;
import java.util.*;
public class DNSClient {
  public static void main(String[] a) throws Exception {
    Scanner sc = new Scanner(System.in);
    System.out.print("Domain: "); String d = sc.nextLine();
    System.out.print("Frame size (bytes): "); int size = sc.nextInt();
    byte[] data = d.getBytes();
    if (size <= 0 || data.length > size) { System.out.println("Invalid frame size"); return; }
    try (DatagramSocket ds = new DatagramSocket()) {
      ds.setSoTimeout(5000);
      ds.send(new DatagramPacket(data, data.length, InetAddress.getByName("localhost"), 5003));
      byte[] b = new byte[1024];
      DatagramPacket p = new DatagramPacket(b, b.length); ds.receive(p);
      System.out.println("DNS reply: " + new String(p.getData(), 0, p.getLength()));
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Input: example.com; frame size: 20. Output: DNS reply: 93.184.216.34.

TABULATION / CALCULATION

Payload size = number of bytes in the domain string. If payload size exceeds the user-entered frame size, the client rejects the request.

RESULT

Thus, a basic DNS request and response was simulated using UDP datagram sockets.

VIVA-VOCE

Question	Answer
What is DNS?	Domain Name System; it maps domain names to IP addresses.
Why is UDP used by DNS?	Traditional DNS queries commonly use UDP because it has low overhead.
What is a datagram?	A self-contained packet sent using a connectionless protocol.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: The example uses a hard-coded local mapping for learning. A real DNS resolver has caching, record types, recursive/iterative lookup, and error codes.

EXPERIMENT 05: TCP File Transfer – User-Requested File Download and FTP Credential Capture

AIM

To transfer a file requested by a client using TCP. Also, to understand how FTP credentials can be observed in an authorized Wireshark lab capture.

PRINCIPLE

TCP provides reliable byte-stream transfer. Traditional FTP sends control information separately from data; standard FTP authentication is not encrypted, so credentials may be visible in a permitted lab capture. Use only your own test network and accounts.

APPARATUS REQUIRED

JDK 8+, two terminals, sample file, Wireshark, and an isolated lab network.

PROCEDURE

1. Create a test file in the server folder.

2. Start the TCP file server and wait for a request.

3. Run the client and enter the required file name.

4. The server checks the file and sends its contents; the client saves it.

5. For the Wireshark demonstration, capture only traffic generated by your own authorized test FTP session.

6. Use a display filter such as `ftp` and inspect the FTP control conversation; do not capture third-party credentials.

PROGRAM / SIMULATOR WORKFLOW

```
// Use the FileServer.java and FileClient.java programs from Experiment 1.
// They send a requested filename first, then transfer exactly the file's byte length.
// Compile both files and run the server before the client.
// Wireshark: start capture on the lab interface, perform your own FTP login,
// stop capture, apply display filter: ftp, then inspect the control packets.
```

SAMPLE OUTPUT / OBSERVATION

Client requests sample.txt; server reports the file sent; client reports successful download. In an authorized plain-FTP test capture, the FTP control exchange can be identified.

TABULATION / CALCULATION

File integrity check: compare original and received file sizes. Stronger check: compute SHA-256 for both files and compare the hashes. Transfer rate = file size / transfer time.

RESULT

Thus, the requested file was transferred through TCP. The authorized capture demonstrates why plain FTP should be replaced with secure alternatives such as SFTP or FTPS.

VIVA-VOCE

Question	Answer
Why is TCP used for file transfer?	It provides reliable, ordered delivery and retransmission.
What is FTP?	File Transfer Protocol, used to transfer files.
Why should plain FTP be avoided?	Its credentials and data are not encrypted.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Do not sniff public Wi-Fi or other users' credentials. Use a private test network, a disposable account, and instructor-approved capture.

EXPERIMENT 06: Reverse Address Resolution Protocol (RARP) Using UDP

AIM

To simulate RARP by obtaining an IP address for a known hardware/MAC address using a UDP client-server program.

PRINCIPLE

RARP historically allowed a device that knows its MAC address to request its IP address from a RARP server. This educational simulation uses UDP and a local mapping. Actual RARP is a link-layer protocol and is not normally implemented over UDP.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, and two terminal windows.

PROCEDURE

1. Start the RARP server on port 5004.

2. The client enters a MAC address.

3. The client sends the MAC address to the server using UDP.

4. The server searches its local MAC-to-IP table.

5. The server returns the IP address or NOT FOUND.

PROGRAM / SIMULATOR WORKFLOW

```
// RARPServer.java
import java.net.*;
public class RARPServer {
  public static void main(String[] a) throws Exception {
    try (DatagramSocket ds = new DatagramSocket(5004)) {
      byte[] b = new byte[100];
      DatagramPacket p = new DatagramPacket(b, b.length); ds.receive(p);
      String mac = new String(p.getData(), 0, p.getLength()).trim();
      String ip = mac.equalsIgnoreCase("AA:BB:CC:DD:EE:01") ? "192.168.1.10" : "NOT FOUND";
      byte[] r = ip.getBytes();
      ds.send(new DatagramPacket(r, r.length, p.getAddress(), p.getPort()));
    }
  }
}
// RARPClient.java
import java.net.*;
import java.util.*;
public class RARPClient {
  public static void main(String[] a) throws Exception {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter MAC address: "); byte[] b = sc.nextLine().getBytes();
    try (DatagramSocket ds = new DatagramSocket()) {
      ds.setSoTimeout(5000);
      ds.send(new DatagramPacket(b, b.length, InetAddress.getByName("localhost"), 5004));
      byte[] r = new byte[100]; DatagramPacket p = new DatagramPacket(r, r.length);
      ds.receive(p); System.out.println("IP address: " + new String(p.getData(),0,p.getLength()));
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Input MAC: AA:BB:CC:DD:EE:01. Output IP address: 192.168.1.10.

TABULATION / CALCULATION

Lookup result is one IP address for one matching MAC entry. An unknown MAC returns NOT FOUND.

RESULT

Thus, the RARP address-mapping concept was simulated using a UDP request and response.

VIVA-VOCE

Question	Answer
What does RARP find?	It finds an IP address corresponding to a known hardware address.
What is a MAC address?	A link-layer hardware address assigned to a network interface.

Question	Answer
Is this a real RARP packet?	No. It is a UDP-based educational simulation of the mapping idea.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: RARP is obsolete in modern networks; DHCP is commonly used for dynamic IP configuration.

EXPERIMENT 07: Distance Vector Routing – Least-Cost Route

AIM

To find the least-cost route between nodes using the Distance Vector Routing algorithm.

PRINCIPLE

Each router maintains a distance table containing the best-known cost to each destination. Routers exchange distance vectors and update costs using Bellman-Ford: D(x,y) = min over neighbors v of [cost(x,v) + D(v,y)].

APPARATUS REQUIRED

Computer, JDK 8+, and terminal/IDE.

PROCEDURE

1. Enter the number of routers and the cost matrix.
2. Use 0 for the diagonal and a large value (999) for no direct link.
3. Initialize each router's distance vector from its direct-link costs.
4. Repeatedly exchange/update distances using the Bellman-Ford rule.
5. Stop when no value changes; display the final routing table.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class DistanceVector {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Number of nodes: "); int n = sc.nextInt();
    int[][] d = new int[n][n];
    System.out.println("Enter cost matrix (999 = no link):");
    for(int i=0;i<n;i++) for(int j=0;j<n;j++) d[i][j]=sc.nextInt();
    for(int k=0;k<n;k++)
      for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
          if(d[i][k] < 999 && d[k][j] < 999 &&
             d[i][j] > d[i][k] + d[k][j])
            d[i][j] = d[i][k] + d[k][j];
    System.out.println("Least-cost routing table:");
    for(int i=0;i<n;i++) {
      System.out.print("Node "+i+": ");
      for(int j=0;j<n;j++) System.out.print(d[i][j]+" ");
      System.out.println();
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

For matrix [[0,2,7],[2,0,1],[7,1,0]], the final least costs are: node 0 → [0,2,3], node 1 → [2,0,1], node 2 → [3,1,0].

TABULATION / CALCULATION

For node 0 to node 2: direct cost = 7; via node 1 = 2 + 1 = 3. Minimum cost = 3.

RESULT

Thus, the least-cost routes were calculated using iterative distance updates.

VIVA-VOCE

Question	Answer
Which algorithmic idea is used?	Bellman-Ford relaxation.
What is a routing metric?	A value such as hop count, delay, or link cost used to choose a route.
When does the algorithm stop?	When no routing-table entry changes during an iteration.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____

Date: _____	Marks: _____ / 100

Additional note: The code computes all-pairs shortest costs for a static matrix, which demonstrates the Bellman-Ford update rule. Real distance-vector protocols exchange vectors periodically with neighbors.

EXPERIMENT 08: Congestion Control Mechanism Simulation Using Network Simulator

AIM

To study and simulate a congestion-control mechanism and observe its effect on network traffic using a network simulator (NS-2/NS-3).

PRINCIPLE

Congestion occurs when offered traffic exceeds available network capacity. TCP congestion control adjusts the sending rate using mechanisms such as slow start, congestion avoidance, fast retransmit, and fast recovery.

APPARATUS REQUIRED

Computer with NS-2 or NS-3 installed, terminal, and a trace/animation viewer such as NAM (for NS-2).

PROCEDURE

1. Create a simple topology with two end hosts and a bottleneck link.

2. Configure a TCP sender and receiver with a limited link bandwidth and queue size.

3. Generate traffic and run the simulation for a fixed duration.

4. Enable trace output and, if available, animation output.

5. Observe congestion-window changes, packet drops, throughput, and delay.

PROGRAM / SIMULATOR WORKFLOW

```
# NS-2 example script: congestion.tcl
set ns [new Simulator]
set tr [open out.tr w]
$ns trace-all $tr
set n0 [$ns node]; set n1 [$ns node]
$ns duplex-link $n0 $n1 1Mb 20ms DropTail
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n1 $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ns at 0.5 "$ftp start"
$ns at 9.5 "$ftp stop"
$ns at 10.0 "close $tr; exit 0"
$ns run
# Run with: ns congestion.tcl
# Note: This is an NS-2 Tcl simulation script, not Java.
```

SAMPLE OUTPUT / OBSERVATION

The trace file records packet transmission and reception. When the bottleneck queue fills, packet drops may occur; TCP responds by reducing its sending rate.

TABULATION / CALCULATION

Throughput = successfully received data bits / simulation time. Packet loss ratio = dropped packets / total packets sent × 100%. Average delay = sum of packet delays / packets received.

RESULT

Thus, congestion-control behaviour was studied in a network simulator. The simulation showed the effect of a bottleneck and TCP rate adaptation.

VIVA-VOCE

Question	Answer
What is congestion?	A condition where network load exceeds available resources.
What is slow start?	A TCP phase in which the congestion window grows rapidly, approximately exponentially per RTT.
What is packet loss ratio?	Dropped packets divided by sent packets, expressed as a percentage.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: This question explicitly requires NS. Java is not the native language for an NS-2 scenario; the Tcl script is included because it is the executable simulator input. If your lab uses NS-3, the script must be rewritten in C++ or Python.

EXPERIMENT 09: Sliding Window Throughput with Packet Size, Error Rate and RTT

AIM

To examine the throughput performance of a sliding-window protocol by varying packet size, error rate, and round-trip time.

PRINCIPLE

Sliding Window allows multiple frames to be in transit before acknowledgements arrive. Larger windows can improve link utilization. Errors cause retransmissions, while larger RTT reduces throughput when the window is small.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, and calculator/spreadsheet for recording results.

PROCEDURE

1. Enter packet size in bytes, window size, RTT in seconds, and packet error rate.

2. Calculate the ideal window-limited throughput.

3. Estimate the successful fraction of transmitted packets.

4. Repeat for different packet sizes, error rates, and RTT values.

5. Tabulate and compare the throughput.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class SlidingWindow {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Packet size (bytes): "); double bytes=sc.nextDouble();
    System.out.print("Window size (packets): "); double w=sc.nextDouble();
    System.out.print("RTT (seconds): "); double rtt=sc.nextDouble();
    System.out.print("Error rate (0 to 1): "); double e=sc.nextDouble();
    if(bytes<=0 || w<=0 || rtt<=0 || e<0 || e>=1) {
      System.out.println("Invalid input"); return;
    }
    double raw = (bytes*8*w)/rtt;
    double good = raw*(1-e);
    System.out.printf("Ideal throughput: %.2f bps%n", raw);
    System.out.printf("Estimated goodput: %.2f bps%n", good);
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Example: packet size 1000 bytes, window 5, RTT 0.1 s, error rate 0.02. Ideal throughput = 400000 bps; estimated goodput = 392000 bps.

TABULATION / CALCULATION

Ideal throughput = (packet size × 8 × window size) / RTT. Estimated goodput = ideal throughput × (1 – error rate). This is a simplified model and excludes headers, ACK overhead, and retransmission timing.

RESULT

Thus, sliding-window throughput was estimated and compared for different packet sizes, error rates, and RTT values.

VIVA-VOCE

Question	Answer
What is a sliding window?	A flow-control method that permits multiple unacknowledged frames.
What is RTT?	Round-trip time for a packet and its acknowledgement.
How does packet loss affect throughput?	It reduces useful throughput because packets may need retransmission.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____

Date: _____	Marks: _____ / 100

Additional note: Record at least three trials per variable. Change only one parameter at a time so its effect can be identified.

EXPERIMENT 10: Comparison of Distance Vector and Link State Routing

AIM

To compare Distance Vector Routing and Link State Routing based on route cost, convergence, and routing information.

PRINCIPLE

Distance Vector routers share destination-cost vectors with neighbors and use Bellman-Ford updates. Link State routers flood link-state information and independently compute shortest paths using Dijkstra's algorithm.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, and sample network topology.

PROCEDURE

1. Create a weighted network graph using an adjacency matrix.

2. Run the Distance Vector/Bellman-Ford shortest-cost calculation.

3. Run Dijkstra's algorithm from each selected source for link-state routing.

4. Compare the resulting least costs and discuss convergence and control overhead.

5. Record the comparison in a table.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class RoutingCompare {
  static final int INF=999;
  public static void main(String[] a) {
    Scanner sc=new Scanner(System.in);
    System.out.print("Nodes: "); int n=sc.nextInt();
    int[][] g=new int[n][n], dv=new int[n][n];
    System.out.println("Cost matrix (999=no link):");
    for(int i=0;i<n;i++) for(int j=0;j<n;j++) g[i][j]=dv[i][j]=sc.nextInt();
    // Bellman-Ford style all-pairs relaxation
    for(int k=0;k<n;k++) for(int i=0;i<n;i++) for(int j=0;j<n;j++)
      if(dv[i][k]<INF && dv[k][j]<INF)
        dv[i][j]=Math.min(dv[i][j],dv[i][k]+dv[k][j]);
    // Dijkstra from source 0
    int[] d=new int[n]; boolean[] used=new boolean[n];
    Arrays.fill(d,INF); d[0]=0;
    for(int c=0;c<n;c++){
      int u=-1;
      for(int i=0;i<n;i++) if(!used[i] && (u==-1 || d[i]<d[u])) u=i;
      if(u==-1 || d[u]>=INF) break;
      used[u]=true;
      for(int v=0;v<n;v++) if(g[u][v]<INF)
        d[v]=Math.min(d[v],d[u]+g[u][v]);
    }
    System.out.println("DV costs from node 0: "+Arrays.toString(dv[0]));
    System.out.println("Link-state (Dijkstra) from node 0: "+Arrays.toString(d));
  }
}
```

SAMPLE OUTPUT / OBSERVATION

For the same static topology, both methods should produce the same shortest-path costs when inputs are valid and convergence is complete.

TABULATION / CALCULATION

Compare cost difference = |Distance Vector cost – Link State cost| for each destination. For a stable graph, expected difference = 0.

RESULT

Thus, Distance Vector and Link State routing calculations were compared. Both find least-cost routes, but they differ in information exchange and computation.

VIVA-VOCE

Question	Answer
Which algorithm is used by link state?	Dijkstra's shortest-path algorithm.
Which algorithm is associated with distance vector?	Bellman-Ford algorithm.
Name a link-state protocol.	OSPF.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: The Java program demonstrates the core route calculations. It does not emulate protocol message flooding or real convergence timing.

EXPERIMENT 11: Error Detection and Correction Techniques

AIM

To implement a basic error-detection/correction demonstration and examine a message received with and without an error.

PRINCIPLE

Error detection adds redundant bits so a receiver can detect corruption. Parity detects some errors; Hamming code can correct a single-bit error. This experiment demonstrates even parity and single-bit correction using a Hamming(7,4) code.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE.

PROCEDURE

1. Enter four data bits.
2. Place data bits in Hamming positions 3, 5, 6, and 7.
3. Calculate parity bits for positions 1, 2, and 4 using even parity.
4. Optionally flip one bit to simulate a transmission error.
5. Calculate the syndrome at the receiver and correct the indicated bit.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class Hamming74 {
  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    System.out.print("Enter 4 data bits (e.g. 1011): ");
    String s=sc.next();
    if(!s.matches("[01]{4}")) { System.out.println("Invalid data"); return; }
    int[] b=new int[8];
    b[3]=s.charAt(0)-'0'; b[5]=s.charAt(1)-'0';
    b[6]=s.charAt(2)-'0'; b[7]=s.charAt(3)-'0';
    b[1]=b[3]^b[5]^b[7];
    b[2]=b[3]^b[6]^b[7];
    b[4]=b[5]^b[6]^b[7];
    System.out.print("Code (positions 1..7): ");
    for(int i=1;i<=7;i++) System.out.print(b[i]);
    System.out.println();
    System.out.print("Enter error position (0=no error, 1..7): ");
    int e=sc.nextInt(); if(e>=1 && e<=7) b[e]^=1;
    int s1=b[1]^b[3]^b[5]^b[7];
    int s2=b[2]^b[3]^b[6]^b[7];
    int s4=b[4]^b[5]^b[6]^b[7];
    int pos=s1+2*s2+4*s4;
    System.out.println("Syndrome/error position: "+pos);
    if(pos>0) b[pos]^=1;
    System.out.print("Corrected code: ");
    for(int i=1;i<=7;i++) System.out.print(b[i]);
  }
}
```

SAMPLE OUTPUT / OBSERVATION

For data 1011, the generated Hamming code is displayed. If bit position 5 is flipped, the receiver calculates syndrome 5 and corrects bit 5.

TABULATION / CALCULATION

Syndrome position = S1 + 2S2 + 4S4. A zero syndrome means no single-bit error was detected; a value 1–7 identifies the bit to correct.

RESULT

Thus, Hamming(7,4) error detection and single-bit correction were demonstrated.

VIVA-VOCE

Question	Answer
What is redundancy?	Extra bits added to data for error detection or correction.
What is a syndrome?	A set of check results used to locate an error in a codeword.

Question	Answer
How many bits can Hamming(7,4) correct?	One-bit errors.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Basic Hamming(7,4) cannot reliably correct multiple-bit errors. Extended Hamming codes add an overall parity bit for SECDED.

EXPERIMENT 12: CRC Error Detection Between Source and Destination

AIM

To simulate CRC error detection between a source and destination and identify whether the received frame contains an error.

PRINCIPLE

Cyclic Redundancy Check treats a bit string as a polynomial and performs modulo-2 division using XOR. The sender appends the remainder; the receiver divides the received codeword by the same generator. A zero remainder indicates no error detected.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, or a network simulator if specifically required by the lab.

PROCEDURE

1. Enter the binary data and generator polynomial as bit strings.

2. Append generator length minus one zero bits to the data.

3. Perform modulo-2 division using XOR to calculate the CRC remainder.

4. Append the remainder to the original data to form the transmitted codeword.

5. At the receiver, divide the received codeword by the same generator and inspect the remainder.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class CRC {
  static String divide(String bits, String gen) {
    char[] a=bits.toCharArray(); int n=gen.length();
    for(int i=0;i<=a.length-n;i++) if(a[i]=='1')
      for(int j=0;j<n;j++) a[i+j]=(a[i+j]==gen.charAt(j))?'0':'1';
    return new String(a, a.length-n+1, n-1);
  }
  public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    System.out.print("Data bits: "); String data=sc.next();
    System.out.print("Generator bits: "); String gen=sc.next();
    if(!data.matches("[01]+") || !gen.matches("1[01]+")) {
      System.out.println("Invalid binary input"); return;
    }
    String zeros="0".repeat(gen.length()-1);
    String rem=divide(data+zeros,gen);
    String code=data+rem;
    System.out.println("CRC remainder: "+rem);
    System.out.println("Transmitted codeword: "+code);
    System.out.print("Received codeword (enter same to test no error): ");
    String recv=sc.next();
    String check=divide(recv,gen);
    System.out.println("Receiver remainder: "+check);
    System.out.println(check.matches("0+")?"No error detected":"Error detected");
  }
}
```

SAMPLE OUTPUT / OBSERVATION

The sender prints the CRC remainder and codeword. If the received codeword is unchanged, the receiver remainder is all zeros and the output is No error detected. Flip a bit to test error detection.

TABULATION / CALCULATION

CRC remainder length = generator length – 1. The transmitted codeword length = data length + CRC remainder length.

RESULT

Thus, CRC generation and receiver-side error detection were simulated using modulo-2 division.

VIVA-VOCE

Question	Answer
What operation is used in CRC division?	XOR (modulo-2 subtraction).
What does a zero remainder indicate?	No error was detected by the CRC check.

Question	Answer
Can CRC correct an error?	CRC detects errors; it does not itself correct them.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Java String.repeat requires Java 11+. For JDK 8, replace `"0".repeat(n)` with a loop that appends n zero characters.

EXPERIMENT 13: Simulation of ARP Protocol for Server and Client

AIM

To simulate the Address Resolution Protocol (ARP) process that maps an IPv4 address to a MAC address.

PRINCIPLE

ARP is used on an IPv4 local network to discover the MAC address associated with a target IPv4 address. A host broadcasts an ARP request; the owner of the IP replies with its MAC address. This Java program simulates the lookup using a local table.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE, and two terminal windows.

PROCEDURE

1. Start the ARP server with a local IP-to-MAC mapping table.

2. Run the client and enter an IPv4 address.

3. The client sends the IP address to the server over UDP for this simulation.

4. The server searches its table and replies with the MAC address or NOT FOUND.

5. The client displays the mapping.

PROGRAM / SIMULATOR WORKFLOW

```
// ARPServer.java
import java.net.*;
public class ARPServer {
  public static void main(String[] a) throws Exception {
    try(DatagramSocket ds=new DatagramSocket(5005)) {
      byte[] b=new byte[100]; DatagramPacket p=new DatagramPacket(b,b.length);
      ds.receive(p); String ip=new String(p.getData(),0,p.getLength()).trim();
      String mac=ip.equals("192.168.1.10")?"AA:BB:CC:DD:EE:01":"NOT FOUND";
      byte[] r=mac.getBytes();
      ds.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort()));
    }
  }
}
// ARPClient.java
import java.net.*; import java.util.*;
public class ARPClient {
  public static void main(String[] a) throws Exception {
    Scanner sc=new Scanner(System.in);
    System.out.print("Enter IP address: "); byte[] b=sc.nextLine().getBytes();
    try(DatagramSocket ds=new DatagramSocket()) {
      ds.setSoTimeout(5000);
      ds.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5005));
      byte[] r=new byte[100]; DatagramPacket p=new DatagramPacket(r,r.length);
      ds.receive(p); System.out.println("MAC address: "+new String(p.getData(),0,p.getLength()));
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Input IP: 192.168.1.10. Output MAC address: AA:BB:CC:DD:EE:01.

TABULATION / CALCULATION

One matching IP table entry returns one MAC address. An unknown IP returns NOT FOUND.

RESULT

Thus, the ARP address-resolution concept was simulated using a client-server request and response.

VIVA-VOCE

Question	Answer
What does ARP map?	An IPv4 address to a MAC address on a local network.
What is an ARP request?	A broadcast query asking which device owns an IPv4 address.
What is an ARP cache?	A temporary table storing recently learned IP-to-MAC mappings.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: This is an application-level simulation using UDP, not a real Ethernet ARP frame. Real ARP operates directly over the local link.

EXPERIMENT 14: Datagram Socket Client/Server Messaging Using UDP

AIM

To implement a datagram socket program in which a message typed at the server side is displayed at the client side.

PRINCIPLE

UDP is connectionless. A DatagramSocket sends and receives DatagramPackets containing data and addressing information. No connection setup is required.

APPARATUS REQUIRED

Computer, JDK 8+, and two terminal windows.

PROCEDURE

1. Start the UDP client so it waits on port 5006.

2. Start the UDP server and type a message.

3. The server sends the message as a datagram to the client IP and port.

4. The client receives and displays the message.

5. Repeat by sending additional messages if the program is extended.

PROGRAM / SIMULATOR WORKFLOW

```
// UDPClient.java
import java.net.*;
public class UDPClient {
  public static void main(String[] a) throws Exception {
    try(DatagramSocket ds=new DatagramSocket(5006)) {
      byte[] b=new byte[1024];
      DatagramPacket p=new DatagramPacket(b,b.length);
      System.out.println("Client waiting...");
      ds.receive(p);
      System.out.println("Server says: "+new String(p.getData(),0,p.getLength()));
    }
  }
}
// UDPServer.java
import java.net.*; import java.util.*;
public class UDPServer {
  public static void main(String[] a) throws Exception {
    Scanner sc=new Scanner(System.in);
    System.out.print("Enter message: "); byte[] b=sc.nextLine().getBytes();
    try(DatagramSocket ds=new DatagramSocket()) {
      ds.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5006));
      System.out.println("Message sent");
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Client waiting... Server says: Hello from UDP server. Server: Message sent.

TABULATION / CALCULATION

Datagram payload length = number of bytes in the message. UDP does not guarantee delivery, ordering, or retransmission.

RESULT

Thus, a message typed at the server was sent using UDP and displayed at the client.

VIVA-VOCE

Question	Answer
Is UDP connection-oriented?	No, it is connectionless.
What is DatagramPacket?	A packet object that stores data and destination/source information.
Does UDP guarantee delivery?	No.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Start the client first, then run the server. For two different computers, replace localhost with the client's LAN IP address.

EXPERIMENT 15: Client Program for Simple Mail Transfer Protocol (SMTP)

AIM

To understand the SMTP client process for sending an email message to a mail server.

PRINCIPLE

SMTP is an application-layer protocol used to transfer outgoing email. A client communicates with an SMTP server using commands such as EHLO, MAIL FROM, RCPT TO, DATA, and QUIT. Modern providers require TLS and authentication.

APPARATUS REQUIRED

Computer, JDK 8+, internet connection, and an authorized SMTP test server/account. A mail library such as Jakarta Mail is recommended for real email.

PROCEDURE

1. Configure an SMTP server hostname and port supplied by the mail provider or lab.

2. Establish a TCP connection; use STARTTLS/implicit TLS as required.

3. Send EHLO and read the server response.

4. Authenticate securely if required, then issue MAIL FROM and RCPT TO.

5. Send DATA, message headers, body, and the terminating dot; then QUIT.

PROGRAM / SIMULATOR WORKFLOW

```
// Educational SMTP command-flow example only.
// Real providers normally require TLS and authentication.
// Use a provider-approved test server; do not hard-code passwords.
import java.io.*;
import java.net.*;
public class SMTPClient {
  public static void main(String[] args) throws Exception {
    String host = "localhost"; int port = 25; // lab SMTP server
    try(Socket s=new Socket(host,port);
        BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out=new PrintWriter(s.getOutputStream(),true)) {
      System.out.println(in.readLine());            // 220 greeting
      out.println("EHLO localhost");
      String line; while((line=in.readLine())!=null) {
        System.out.println(line);
        if(line.length()>=4 && line.charAt(3)==' ') break;
      }
      // Continue MAIL FROM / RCPT TO / DATA only on an authorized lab server.
      out.println("QUIT");
      System.out.println(in.readLine());
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

The client connects to the lab SMTP server, reads its greeting, sends EHLO, displays the server capabilities, and issues QUIT.

TABULATION / CALCULATION

SMTP response codes: 2xx = success, 3xx = further input required, 4xx = temporary failure, 5xx = permanent failure.

RESULT

Thus, the basic SMTP client connection and command exchange were demonstrated. Actual email delivery requires a correctly configured authenticated TLS session.

VIVA-VOCE

Question	Answer
What is SMTP?	Simple Mail Transfer Protocol, used to send email.
What does EHLO do?	Identifies the client and requests server extensions.
What is the purpose of TLS?	It encrypts the connection and protects credentials/data in transit.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: For production or common public mail providers, use Jakarta Mail (Angus Mail) with TLS and an app password/OAuth as supported. Never store account passwords directly in source code.

EXPERIMENT 16: RARP Protocol Simulation for Server and Client

AIM

To write a client-server program simulating RARP, where a client sends its MAC address and receives the corresponding IP address.

PRINCIPLE

RARP is the reverse mapping concept: MAC address to IP address. This experiment is similar to Experiment 6 but is written as a request-response client/server demonstration.

APPARATUS REQUIRED

Computer, JDK 8+, and two terminal windows.

PROCEDURE

1. Run the RARP server on the selected UDP port.

2. Run the client and enter a MAC address from the server's mapping table.

3. The client sends the MAC address to the server.

4. The server checks the mapping and returns the IP address.

5. The client prints the returned IP address.

PROGRAM / SIMULATOR WORKFLOW

```
// This is a compact alternative implementation of Experiment 6.
// Server and client use UDP port 5016.
import java.net.*; import java.util.*;
class RARPServer16 {
  public static void main(String[] a) throws Exception {
    try(DatagramSocket ds=new DatagramSocket(5016)) {
      byte[] b=new byte[100]; DatagramPacket p=new DatagramPacket(b,b.length);
      ds.receive(p);
      String mac=new String(p.getData(),0,p.getLength()).trim();
      String ip=mac.equalsIgnoreCase("00:11:22:33:44:55")?"10.0.0.5":"NOT FOUND";
      byte[] r=ip.getBytes();
      ds.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort()));
    }
  }
}
class RARPClient16 {
  public static void main(String[] a) throws Exception {
    Scanner sc=new Scanner(System.in);
    System.out.print("MAC: "); byte[] b=sc.nextLine().getBytes();
    try(DatagramSocket ds=new DatagramSocket()) {
      ds.setSoTimeout(5000);
      ds.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5016));
      byte[] r=new byte[100]; DatagramPacket p=new DatagramPacket(r,r.length);
      ds.receive(p); System.out.println("Mapped IP: "+new String(p.getData(),0,p.getLength()));
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Input MAC: 00:11:22:33:44:55. Output: Mapped IP: 10.0.0.5.

TABULATION / CALCULATION

Successful lookup = one MAC-to-IP table match. Unmatched MAC address = NOT FOUND.

RESULT

Thus, RARP client-server mapping was simulated successfully.

VIVA-VOCE

Question	Answer
What is the reverse mapping in RARP?	MAC address to IP address.
Which modern protocol replaced the need for RARP?	DHCP is commonly used for host configuration.
What happens for an unknown MAC?	The server returns NOT FOUND.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: As with Experiment 6, this is a UDP simulation of the mapping concept, not an actual RARP protocol frame.

EXPERIMENT 17: HTTP Web Client Using TCP Sockets and Network-Layer Packet Investigation

AIM

To write an HTTP web client that downloads a webpage using TCP sockets and to investigate packets exchanged at the network layer using a simulator or packet analyzer.

PRINCIPLE

HTTP request/response messages are carried over TCP for HTTP/1.1. Network-layer packets use IP addressing and are forwarded between networks. Packet inspection can reveal source/destination IP, protocol, packet length, and timing.

APPARATUS REQUIRED

Computer, JDK 8+, internet connection, Wireshark or approved network simulator.

PROCEDURE

1. Choose a test HTTP host that supports plain HTTP on port 80.
2. Open a TCP socket to the host.
3. Send a valid HTTP GET request with the Host header and Connection: close.
4. Read the response and save it to a file.
5. Capture the test traffic in Wireshark; inspect IP source/destination, TCP ports, and HTTP request/response.
6. Use HTTPS for real browsing; this simple program is only for a permitted HTTP test.

PROGRAM / SIMULATOR WORKFLOW

```
import java.io.*;
import java.net.*;
public class HTTPClient {
  public static void main(String[] args) throws Exception {
    String host=args.length>0?args[0]:"example.com";
    try(Socket s=new Socket(host,80);
        PrintWriter out=new PrintWriter(s.getOutputStream(),true);
        BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter file=new PrintWriter("downloaded_page.html")) {
      out.print("GET / HTTP/1.1\r\nHost: "+host+
                "\r\nConnection: close\r\n\r\n"); out.flush();
      String line;
      while((line=in.readLine())!=null) file.println(line);
      System.out.println("Saved response to downloaded_page.html");
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

The program connects to the HTTP host and saves the server response to downloaded_page.html. Wireshark shows TCP connection packets and HTTP request/response when the traffic is unencrypted.

TABULATION / CALCULATION

Packet length and count can be read from the capture. Approximate throughput = total captured data bits / capture duration.

RESULT

Thus, a webpage response was downloaded through a TCP socket and network-layer packet fields were examined.

VIVA-VOCE

Question	Answer
What is HTTP?	Hypertext Transfer Protocol, used for web communication.
What is the network layer protocol?	IP (Internet Protocol).
What does the Host header identify?	The website hostname requested by the HTTP client.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Many websites redirect HTTP to HTTPS or block port 80. If the connection fails, use an instructor-provided HTTP test server. This basic code is for HTTP only and does not implement TLS.

EXPERIMENT 18: TCP/UDP Performance Analysis Using Simulation and FTP Capture

AIM

To analyze TCP and UDP performance using a network simulation tool and to identify FTP control traffic in an authorized Wireshark lab capture.

PRINCIPLE

TCP provides reliability, sequencing, flow control, and congestion control. UDP has lower protocol overhead but does not guarantee delivery. Performance can be compared using throughput, delay, and packet loss. Plain FTP control traffic is not encrypted.

APPARATUS REQUIRED

NS-2/NS-3 or another approved simulator, Wireshark, computer, and isolated lab network.

PROCEDURE

1. Create a topology with two end hosts and a bottleneck link.
2. Generate TCP traffic and record throughput, delay, and packet loss.
3. Repeat with UDP traffic under the same topology and offered load.
4. Compare the results using a table.
5. For FTP capture, generate a test FTP session using a disposable account on a private lab server.
6. Capture only that authorized session and filter with `ftp`.

PROGRAM / SIMULATOR WORKFLOW

```
# NS-2 starter topology (Tcl); run in an NS-2 lab environment.
set ns [new Simulator]
set tr [open compare.tr w]
$ns trace-all $tr
set a [$ns node]; set b [$ns node]
$ns duplex-link $a $b 1Mb 10ms DropTail
# TCP flow
set tcp [new Agent/TCP]
set sink [new Agent/TCPSink]
$ns attach-agent $a $tcp; $ns attach-agent $b $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]; $ftp attach-agent $tcp
$ns at 0.5 "$ftp start"; $ns at 4.5 "$ftp stop"
# For UDP, create Agent/UDP + Agent/Null and CBR traffic
# as a second run so the comparison uses the same topology/load.
$ns at 5.0 "close $tr; exit 0"
$ns run
# This simulator input is Tcl, not Java; use NS-3 C++/Python if required.
```

SAMPLE OUTPUT / OBSERVATION

The simulation produces trace data for TCP/UDP flows. TCP adapts its sending rate and retransmits lost data; UDP sends without built-in retransmission. The actual measured values depend on the topology and traffic settings.

TABULATION / CALCULATION

Throughput = received bits / simulation time. Average delay = total end-to-end delay / packets received. Packet loss (%) = lost packets / sent packets × 100%.

RESULT

Thus, TCP and UDP performance metrics were collected and compared. An authorized FTP capture was also examined.

VIVA-VOCE

Question	Answer
Which protocol retransmits lost data by default?	TCP.
Which protocol is connectionless?	UDP.
What does Wireshark do?	Captures and analyzes network packets.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: Do not claim fixed numerical results without running the simulator. Fill the result table with your own measured trace values. Never capture other users' FTP credentials.

EXPERIMENT 19: HDLC Frame – Bit Stuffing and Character Stuffing

AIM

To implement bit stuffing and character/byte stuffing concepts used in data-link framing.

PRINCIPLE

HDLC uses the flag 01111110 to mark frame boundaries. To prevent the flag pattern from appearing inside data, the sender inserts a 0 after every sequence of five consecutive 1 bits. Byte/character stuffing inserts an escape byte before reserved control bytes.

APPARATUS REQUIRED

Computer, JDK 8+, terminal/IDE.

PROCEDURE

1. Enter a binary data string and validate that it contains only 0 and 1.

2. Scan the bits and insert 0 after every five consecutive 1s.

3. Display the stuffed bit stream between HDLC flags.

4. For character stuffing, scan the data bytes and insert an escape character before any reserved flag or escape byte.

5. Display the resulting byte-stuffed data.

PROGRAM / SIMULATOR WORKFLOW

```
import java.util.*;
public class HDLCStuffing {
  static String bitStuff(String s) {
    StringBuilder r=new StringBuilder(); int ones=0;
    for(char c:s.toCharArray()) {
      r.append(c);
      if(c=='1') { ones++; if(ones==5){r.append('0'); ones=0;} }
      else ones=0;
    }
    return "01111110"+r+"01111110";
  }
  static String charStuff(String s) {
    StringBuilder r=new StringBuilder();
    char FLAG='F', ESC='E'; // illustrative reserved characters
    for(char c:s.toCharArray()) {
      if(c==FLAG || c==ESC) r.append(ESC);
      r.append(c);
    }
    return r.toString();
  }
  public static void main(String[] a) {
    Scanner sc=new Scanner(System.in);
    System.out.print("Binary data: "); String bits=sc.next();
    if(!bits.matches("[01]+")) {System.out.println("Invalid bits"); return;}
    System.out.println("Bit stuffed frame: "+bitStuff(bits));
    System.out.print("Text for character stuffing: ");
    sc.nextLine(); String text=sc.nextLine();
    System.out.println("Character stuffed: "+charStuff(text));
  }
}
```

SAMPLE OUTPUT / OBSERVATION

For bit data 111110, the stuffed data becomes 1111100, placed between 01111110 flags. Character stuffing inserts E before each reserved F or E character.

TABULATION / CALCULATION

Number of inserted bit-stuffing zeros = number of times five consecutive 1s are encountered during scanning. Character-stuffing overhead = number of reserved characters in the input.

RESULT

Thus, HDLC bit stuffing and illustrative character/byte stuffing were implemented.

VIVA-VOCE

Question	Answer
What is the HDLC flag?	01111110.
When is a bit 0 inserted?	After five consecutive 1 bits in the data field.
What is byte stuffing?	Adding an escape byte before reserved control bytes.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: The character-stuffing section uses F and E as illustrative flag/escape characters. Actual protocols define their own byte values and escaping rules.

EXPERIMENT 20: TCP Server Reads a String, Displays Day/Time and Replies to Client; TCP Performance

AIM

To create a TCP client-server program in which the server reads a string, displays the current day and time, and sends a reply to the client. Also, to study TCP performance using a simulation tool.

PRINCIPLE

TCP provides a reliable connection between client and server. Java's LocalDateTime can obtain the server's current date and time. Network simulation can measure TCP throughput, delay, and packet loss.

APPARATUS REQUIRED

Computer, JDK 8+, two terminals, and NS-2/NS-3 for the performance part.

PROCEDURE

1. Start the server on port 5020.

2. The client connects and sends a text string.

3. The server reads the string and obtains the current date and time.

4. The server prints the received string and time, then sends a reply.

5. The client displays the reply and closes the connection.

6. For TCP performance, run an approved simulator scenario and calculate throughput, delay, and packet loss.

PROGRAM / SIMULATOR WORKFLOW

```
// TimeServer.java
import java.io.*; import java.net.*; import java.time.*;
public class TimeServer {
  public static void main(String[] a) throws Exception {
    try(ServerSocket ss=new ServerSocket(5020)) {
      try(Socket s=ss.accept();
          BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
          PrintWriter out=new PrintWriter(s.getOutputStream(),true)) {
        String msg=in.readLine();
        String time=LocalDateTime.now().toString();
        System.out.println("Client message: "+msg);
        System.out.println("Day/time: "+time);
        out.println("Received: "+msg+" | Server time: "+time);
      }
    }
  }
}
// TimeClient.java
import java.io.*; import java.net.*;
public class TimeClient {
  public static void main(String[] a) throws Exception {
    try(Socket s=new Socket("localhost",5020);
        BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out=new PrintWriter(s.getOutputStream(),true);
        BufferedReader kb=new BufferedReader(new InputStreamReader(System.in))) {
      System.out.print("Enter message: "); out.println(kb.readLine());
      System.out.println("Server reply: "+in.readLine());
    }
  }
}
```

SAMPLE OUTPUT / OBSERVATION

Client enters: Good morning. Server prints the message and current date/time. Client receives: Received: Good morning | Server time: [current date/time].

TABULATION / CALCULATION

TCP throughput = total successfully received bits / elapsed time. Average delay = sum of packet delays / number of received packets. Packet loss (%) = lost packets / sent packets × 100%.

RESULT

Thus, the server received and displayed the client message with the current day/time and returned a reply. TCP performance metrics can be obtained from the simulator trace.

VIVA-VOCE

Question	Answer
Which Java class provides date and time?	LocalDateTime.
What is a TCP three-way handshake?	SYN, SYN-ACK, and ACK.
Name one TCP performance metric.	Throughput, end-to-end delay, or packet loss.

RECORD / FACULTY SIGNATURE

Student Signature: _____	Staff Signature: _____
Date: _____	Marks: _____ / 100

Additional note: The exact simulator measurements depend on your topology, bandwidth, queue size, packet size, and simulation duration. Record actual values from your run.

FINAL PRACTICAL EXAMINATION MARKS – RECORD SHEET

Component	Maximum Marks	Marks Obtained
Aim / Principle / Apparatus Required / Procedure	20	
Tabulation / Circuit / Program / Drawing	30	
Calculation & Results	30	
Viva-Voce	10	
Record	10	
Total	100	

Faculty Remarks

_____

_____

Faculty Name & Signature: _____

Date: _____